

Software Engineering Learning Curriculum

Duration: 3 Months (Starting March 2026)

Goal: Build a strong foundation for an IT Software Engineering career

Prepared on March 16, 2026

Guiding Principles

How You Will Use AI (Non-Negotiable)

AI (ChatGPT, Claude, etc.) is your teacher, not your coder.

- Never copy-paste AI-generated code into your projects.
- Always write code yourself, by hand, from understanding.
- Use AI the way you would use a knowledgeable mentor. Ask it:
 - *"Explain conceptually how a HashMap works internally."*
 - *"I'm thinking of using an ArrayList here. What are the trade-offs vs a LinkedList?"*
 - *"Why would this approach fail for large datasets?"*
 - *"What is the problem with the other options?"*
 - *"If not this, then what? And why?"*
- After writing your code manually, you may share it with AI for a code review and feedback. Then improve it yourself.

The goal is to build your own mental model, not to produce output fast.

How You Will Use LeetCode

LeetCode runs in parallel throughout all three months, but the goal is not a high score.

- Solve 2–3 easy problems per week.
- For every problem, try multiple approaches before looking at solutions.
- Focus on understanding: Why does this approach work? Why doesn't the other one?
- Categories to rotate through: Arrays, Strings, HashMaps, Sorting, Linked Lists, Stacks/Queues, Basic Trees.
- Track what you attempted and what you learned, not just what you solved.

Month 1: Building the Foundation

Theme: Core Programming + Data Structures using Real-World University Data

Week 1–2: Core Java & Python Fundamentals

Setup:

- Install JDK 17+, set up a Java project (no IDE magic — understand compilation: javac, java)
- Install Python 3.11+, set up a Python project
- Learn to use a simple editor or IDE (VS Code or IntelliJ Community)
- Initialize a Git repository from Day 1. Commit every meaningful change.

Concepts to solidify (in both Java and Python):

- Variables, data types, type casting
- Conditionals (if/else, switch)
- Loops (for, while, for-each)
- Functions/Methods — parameters, return types, scope
- Arrays and basic string operations
- File I/O — reading and writing text files, CSVs
- Object-Oriented Programming: Classes, Objects, Constructors, Encapsulation
- Error handling basics (try-catch in Java, try-except in Python)

Week 2–3: Build Your University Data System (Core Approach)

Step 1: Generate Synthetic Data

Use AI to generate realistic CSV/Excel files with ~100 records each. This is the one permitted use of AI for generation — data, not code.

Create the following datasets:

#	CSV File	Key Columns
1	departments.csv	dept_id, dept_name, building, head_of_dept
2	streams.csv	stream_id, stream_name, dept_id
3	students.csv	student_id, name, stream_id, dept_id, enrollment_year
4	subjects.csv	subject_id, subject_name, dept_id, credits
5	student_marks.csv	student_id, subject_id, semester, marks_obtained, max_marks
6	teachers.csv	teacher_id, name, dept_id,

		specialization
7	class_sessions.csv	session_id, subject_id, teacher_id, day_of_week, start_time, end_time, room
8	teacher_assignments.csv	teacher_id, subject_id, semester

Step 2: Build Entity Models

In both Java and Python, create class/object models for each dataset. For example: a Student object holds id, name, stream, department, enrollment year. A Mark object holds student reference, subject reference, semester, marks. Load CSV rows into lists/arrays of these objects.

Rules for this phase:

- Java: Use only arrays, String.split(), BufferedReader. No ArrayList, no HashMap, no Stream API.
- Python: Use only basic lists, dictionaries, loops, open() and .split(). No pandas, no csv module, no list comprehensions initially.

Step 3: Write Query Programs

Build programs that answer the following (implement each as a separate function/method):

#	Query
1	What is the highest score in a given subject? Which student got it?
2	Which teacher taught that subject?
3	Average, maximum, and minimum score for a given subject
4	Department-wise student count
5	Which department has the most intensive classroom schedule? (most sessions per week)
6	List all subjects a given student is enrolled in with their marks
7	Find all teachers in a given department and their assigned subjects
8	Which day of the week has the most packed schedule across all departments?
9	Rank students by overall percentage across all subjects
10	Find teachers who teach more than 3

Data Structure Learning (connect theory to practice):

As you solve these, you will naturally encounter and understand:

- Arrays — storing and iterating over records
- Searching — linear search through records
- Sorting — sorting students by marks, teachers by session count
- Nested loops — joining data across CSVs (e.g., student → marks → subject → teacher)
- String manipulation — parsing CSV lines, formatting output

Week 3–4: Redo Everything with Advanced Tools

Now solve the exact same problems using powerful built-in tools:

Java — Collection Framework & Streams:

- Replace arrays with ArrayList, HashMap, HashSet
- Use Collections.sort() with Comparator
- Use Java Streams: filter(), map(), collect(), groupingBy(), summarizingInt()
- Use Optional for null safety

Python — Built-in Functions & Pandas:

- Use list comprehensions, sorted() with key=lambda, filter(), map()
- Use collections.Counter, collections.defaultdict
- Learn Pandas fundamentals:
 - pd.read_csv() — loading data
 - df.head(), df.describe(), df.info() — exploration
 - df[condition] — filtering
 - df.groupby() — aggregation
 - df.merge() — joining datasets (like SQL joins)
 - df.sort_values(), df.apply() — transformation
 - df.pivot_table() — summarization

Why this matters: Pandas covers virtually everything Microsoft Excel can do, but programmatically. This skill will be invaluable for handling JSON, CSV, and complex data structures in any future role.

Month 1 Checkpoint

By end of Month 1, you should be able to:

- ☐ Read any CSV and load it into Java/Python objects from scratch
- ☐ Write basic data processing logic without any libraries
- ☐ Use Java Collections and Streams fluently
- ☐ Use Python Pandas for data manipulation
- ☐ Explain the difference between an Array, ArrayList, HashMap, and HashSet
- ☐ Have a Git repository with clean, well-organized code and meaningful commits

Month 2: Industry-Grade Data & Databases

Theme: Complex E-commerce Data + Relational & NoSQL Databases

Note: Detailed plan will be finalized based on Month 1 progress. Below is the high-level roadmap.

The E-commerce Dataset

You will work with a realistic e-commerce data model:

Entity	Key Attributes
Customers	customer_id, name, email, address, registration_date
Products	product_id, name, category, price, stock_quantity
Orders	order_id, customer_id, order_date, status, total_amount
Order Items	order_id, product_id, quantity, unit_price
Payments	payment_id, order_id, payment_method, amount, status, payment_date
Promotions	promo_id, code, discount_type, discount_value, valid_from, valid_to
Deliveries	delivery_id, order_id, status, estimated_date, actual_date, carrier

Database World

PostgreSQL (Relational Database):

- Install PostgreSQL locally
- Learn SQL fundamentals: CREATE TABLE, INSERT, SELECT, JOIN, GROUP BY, HAVING, ORDER BY
- Design proper tables with primary keys, foreign keys, and constraints
- Write complex queries: subqueries, aggregations, window functions
- Connect Java (JDBC) and Python (psycopg2) to PostgreSQL
- Perform the same e-commerce queries both in raw SQL and through code

Redis (NoSQL / In-Memory Store):

- Understand when and why you use Redis vs PostgreSQL
- Learn Redis data structures: Strings, Lists, Sets, Hashes, Sorted Sets
- Use cases: caching, session storage, leaderboards, rate limiting
- Connect Java (Jedis) and Python (redis-py) to Redis

Month 2 Problem Statements (Preview)

- Top-selling products by category
- Customer lifetime value calculation
- Order fulfillment rate and average delivery time
- Revenue trends by month
- Promotion effectiveness analysis
- Identify customers at risk of churn
- Inventory restocking recommendations

Month 3: Full Stack Application Development

Theme: Build a Real Application from the Ground Up

Note: Detailed plan will be finalized based on Month 1 and 2 progress. Below is the high-level roadmap.

Backend: Microservices

Java Spring Boot:

- Build REST APIs for the University and E-commerce data
- Learn: Controllers, Services, Repositories, DTOs
- Connect to the same PostgreSQL database from Month 2
- Implement CRUD operations
- Learn request/response lifecycle, HTTP methods, status codes

Python FastAPI:

- Build the same set of APIs in FastAPI
- Learn: route decorators, Pydantic models, dependency injection
- Connect to the same PostgreSQL database
- Understand async fundamentals

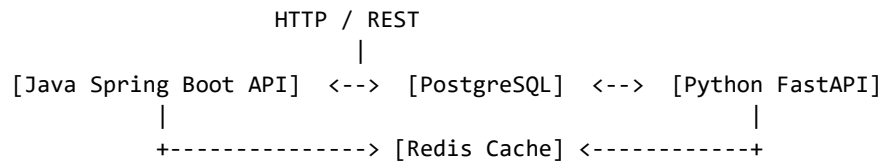
Both services will talk to the same PostgreSQL database — this teaches you that databases are independent of application code.

Frontend: React.js

- Learn HTML, CSS, JavaScript fundamentals (if not already known)
- Set up a React project
- Build components for:
 - University dashboard (student records, department stats, timetable view)
 - E-commerce storefront (product listing, order history, customer dashboard)
- Connect React frontend to your Spring Boot / FastAPI backends
- Learn: components, props, state, hooks, API calls with fetch/axios

Architecture Overview

[React Frontend]
|



Weekly Rhythm (All 3 Months)

Day	Activity
Mon – Fri	2–3 hours focused coding (the monthly project work)
Saturday	1 hour LeetCode (2–3 problems, focus on understanding)
Sunday	Review the week: revisit what was hard, clean up code, update Git

Tools & Setup

Tool	Purpose
VS Code or IntelliJ IDEA Community	Code editor / IDE
Git + GitHub	Version control from Day 1
PostgreSQL	Relational database (Month 2+)
Redis	In-memory data store (Month 2+)
Node.js + npm	React development (Month 3)
Postman	API testing (Month 2–3)
AI Chat (Claude/ChatGPT)	Conceptual learning and code review only

Final Note

This curriculum is designed to build your skills from the ground up. The temptation to skip ahead or use shortcuts will be strong. Resist it.

The students who succeed in software engineering interviews and careers are not the ones who memorized the most solutions — they are the ones who can think through a problem, understand trade-offs, and write clean code from scratch.

Every line of code you write manually in Month 1 will pay dividends in Month 3 when you are building real applications. Trust the process.